Principal Engineering Review

Dojo Coding-Agent Evaluation System | System & Trust Model

1. End-to-End Evaluation Data Flow

The evaluation pipeline transforms an approved task specification and a candidate workspace snapshot into an isolated test execution receipt and a deterministic scoring verdict.

```
[1. Approved Task Identity & Frozen Manifests]
       |  (load_contract via input_guard_v2 + SHA-256 verification)
       ├──► Original Base Snapshot ──► [2. Solver Environment (DockerSandbox)]
       |                                       | (Candidate edits & runs tests)
       |                                       ▼
       |                        [3. Candidate Snapshot + Unified Diff]
       ▼                                       |
[4. Restored Base+Hidden-Tests]  ◄── [prepare_candidate_reference]
       |  (Verify make_patch; filter candidate files via editable_source)
       ▼
[5. Prepared Evaluation Archive] + [Control JSON]
       |  (_bounded_run -> docker run --network=none --read-only --user=65534:65534)
       ▼
[6. Container Python Harness (/usr/local/bin/python -I -c ...)]
       ├──► Extract archive (enforce path, count <= 2000, size <= 16 MiB)
       ├──► Offline Install (legacy-v1 only: pip install --no-deps)
       ├──► Redirect sys.stdout/sys.stderr to /dev/null
       └──► Run pytest.main(..., plugins=[receipt])
              |
[7. Receipt Transport & Parsing]
       |  (Container prints DOJO_CONTRACT_RECEIPT=...; host parses)
       ▼
[8. Final Scoring Decision (judge_receipt)] -> DockerScore(status, reason, ...)
```

2. Trust and Verification Matrix

Stage	Checks Performed	Assumptions Relied Upon	Info Reaching Solver
1. Task Identity	Verifies `record_sha256`, baseline SHA-256, fix snapshot SHA-256, and checks all 4 frozen construction runs via `judge_receipt`.	`input_guard_v2` strips Git overrides & verifies identity. `ChangedFile.is_test` accurately classifies test paths.	Issue text and `original_archive` (prior to hidden test overlay).
2. Solver Env	Enforces timeout, caps workspace at 2000 files / 16MiB, truncates stdout to 3000 bytes.	`safe_path` logic; `snapshot_archive` produces deterministic tar streams.	`FakeStep` with truncated output. **Never** receives hidden tests or contracts.
3. Snapshot & Diff	Checks `make_patch == patch`. Transfers only files matching `editable_source`.	`ChangedFile.is_test` blocks all hidden test files from candidate override.	None (runs post-solver).
4. Container Setup	Inspects local image ID. Runs with strict cgroups, tmpfs, and UID 65534.	Docker daemon honors cgroups. `flit_core` builds offline safely.	None.

3. Adversarial Coverage Matrix

Review Area	Status	Assessment Summary
Archive paths, canonicalization, file types & size limits	Adequately Checked	Host checks `len <= 20 MiB`, `_valid_name`, counts/size. Container enforces strict path limits, ASCII >= 32, max 2000 files / 16 MiB.
Snapshot/diff agreement & source additions/deletions	**Actionable Defect**	`make_patch` raises `ValueError` on 0-byte file creation/deletion and crashes on non-UTF-8 bytes (Finding 3).
Test files, injected hooks, config & packaging	Adequately Checked	`editable_source` tightly restricts candidate edits to `src/` or `more_itertools/` and explicitly drops `conftest.py` and root configs.
Package imports, metadata & offline build behavior	**Actionable Defect**	Candidate syntax/import bugs break `flit_core`, causing `pip install` to crash the container (`RuntimeError`). `sys.path` ordering shadows wheels (Finding 4).
Pytest collection, setup/teardown, hooks & disabled plugins	**Actionable Defect**	Candidate bugs causing collection failures (exit code 2) or fixture setup errors are misclassified as `invalid_evidence` instead of `failed` (Finding 1).
Missing, extra, duplicate, failed or skipped methods	**Actionable Defect**	`legacy-v1` double-counts tests that pass but then skip in teardown, leading to `invalid_evidence` rejections (Finding 5).
Expected skip drift & nested subtest aggregation	Adequately Checked	`ReceiptV2` correctly handles subtests. `judge_receipt` strictly enforces exact `stable_skips` sets without drift.
Malformed receipts, duplicate keys, noisy output & forged output	**Actionable Defect**	Harness leaves OS FD 1 open. Candidate `os.write(1)` or subprocesses corrupt the receipt stream, crashing evaluation (Finding 2). Interpreter shared.
Premature exit, process signals, descendants & background activity	**Actionable Defect**	Candidate `sys.exit(1)` triggers container `RuntimeError`. Lingering child processes hold stdout pipes open after PID 1 exits (Finding 6).
Stdout/stderr limits, deadlines, image selection & cleanup	**Actionable Defect**	`_bounded_run` CLI kill delays `docker rm` by up to 8s. A single transient `docker rm` failure aborts cleanup (Finding 6).
Evaluator-to-solver info leakage	Adequately Checked	`DockerSandbox` only receives base archive. Evaluator contracts, receipts, and hidden tests are entirely isolated from the solver process.
Compatibility with frozen construction evidence	Adequately Checked	`_make_contract` replays and strictly enforces exact matches on all 4 frozen construction runs during load.

4. Detailed Actionable Findings

Finding 1: Candidate Failures Misclassified as `invalid_evidence` or `RuntimeError`

Severity: **High** — Violates Objective 3. In automated benchmark evaluation, syntax errors, missing imports, or `sys.exit()` calls are common solver mistakes. Raising `RuntimeError` halts benchmark runs.

Mechanics: If a candidate introduces a `SyntaxError`, `pytest.main` returns `ExitCode.INTERRUPTED` (2). `judge_receipt` rejects `exit_code == 2` as `invalid_evidence`. If a candidate breaks `src/click/__init__.py`, `pip install` raises `CalledProcessError`, the container exits with code 1, and the harness raises `RuntimeError('Scoring container or offline installation failed')`, conflating a candidate bug with a Docker failure.

```
# UNEXECUTED Reproduction
def test_candidate_syntax_error_is_scored_as_failed(task):
    contract, source, base = task
    bad_files = {**base, source: b"def add(a, b):\n    return (\n"} # SyntaxError
    score = run_development_score(contract, snapshot_archive(bad_files), patch)
    assert score.status == "failed" # Currently returns invalid_evidence or raises
```

Fix: Catch candidate-induced `pip install` failures and return a valid receipt marked failed. Classify candidate `exit_code == 2` or setup/collection errors as `failed` during candidate evaluation (while preserving strict structural checks for frozen construction loading).

Finding 2: OS-Level FD 1 Leakage Causes False Crashes

Severity: **High** — Violates Objective 2. Legitimate successful fixes are rejected. Relevant for `pallets/click` (a CLI framework).

Mechanics: `contextlib.redirect_stdout` only reassigns Python's `sys.stdout`. OS file descriptor `1` remains connected to the host pipe. If any candidate code or test spawns a subprocess (e.g. `click.edit`) or writes directly via `os.write(1, ...)`, those bytes appear before `DOJO_CONTRACT_RECEIPT=`. `parse_receipt` sees multiple lines and raises `RuntimeError('Missing, noisy or ambiguous scoring receipt')`.

```
# UNEXECUTED Reproduction
def test_os_fd1_write_does_not_corrupt_receipt(task):
    contract, source, base = task
    noisy_fix = b"import os\ndef add(a, b):\n    os.write(1, b'noise\\n')\n    return a+b\n"
    score = run_development_score(contract, snapshot_archive({**base, source: noisy_fix}), patch)
    assert score.status == "passed" # Currently crashes with RuntimeError
```

Fix: Duplicate FD 1 (`receipt_fd = os.dup(1)`), redirect FDs 1 and 2 to `/dev/null` via `os.dup2`, and write the receipt exclusively to `receipt_fd`.

Finding 3: `make_patch` Rejects Empty File Creation/Deletion

Severity: Medium — Violates Objective 2. Python packages routinely use `0`-byte `__init__.py` files. If the solver touches an empty file, the sandbox bricks permanently.

Mechanics: In `make_patch(base, current)`, if a candidate creates an empty file, both `old` and `new` evaluate to `b""`. The code explicitly checks `if not old and not new: raise ValueError("Empty-file creation/deletion needs a later patch exporter")`. This uncaught exception inside `DockerSandbox.execute()` permanently sets `self.failed = True`. Additionally, decoding patches without `UnicodeDecodeError` handling bricks the sandbox on binary/malformed output.

```
# UNEXECUTED Reproduction
def test_empty_init_py_creation_does_not_fail(task):
    contract, source, base = task
    new_init = contract.source_prefix + "subpkg/__init__.py"
    files = {**base, source: FIXED, new_init: b""}
    patch = make_patch(base, files) # Currently raises ValueError!
    assert run_development_score(contract, snapshot_archive(files), patch).status == "passed"
```

Fix: Bypass `difflib` for empty file transitions and manually output standard headers (`--- /dev/null\n+++ b/...`). Catch `UnicodeDecodeError` and raise cleanly.

Finding 4: `sys.path` Shadows Offline-Installed Wheels

Severity: Medium — Modifies import behavior relative to standard installation. In `INSTALL` for `legacy-v1`, `pip install --target=/tmp/site` runs, and `/tmp/site` is added to `sys.path`. However, immediately after, `/tmp/work/src` and `/tmp/work` are inserted at `sys.path[0]`, pushing the installed wheel down. The test suite imports raw source instead of the wheel, although it resolves `.dist-info` metadata from `/tmp/site`.

Fix Constraint: Because `sys.path` ordering is embedded in the frozen `legacy-v1` code used to generate the 34 Click construction tasks, altering it now breaks parity with frozen results. Document this behavior as a fixed characteristic of the v1 evaluation environment.

Finding 5: `legacy-v1` Double-Counts Teardown Skips

Severity: Low / Medium — The legacy `Receipt` class appends a test to `self.passed` during the `call` phase, but if a fixture finalizer subsequently calls `pytest.skip()` during `teardown`, it also appends to `self.skipped` without checking `report.when`. This results in `len(flat) != collected`, triggering a false `invalid_evidence` verdict. (Preserve existing harness due to frozen constraints; handle edge cases in scoring logic if required).

Finding 6: Background Descendant Pipe Leaks & Cleanup Timeout

Severity: **Medium** — Violates Objective 5. `_bounded_run` kills the local `docker` CLI client on timeout, but the container remains running in the background until `docker rm --force` executes.

Mechanics: Inside the container, PID 1 (Python) exits upon printing the receipt. The kernel sends SIGKILL to the PID namespace. However, if a child subprocess writes to the inherited stdout FD *before* PID 1 exits, it pollutes the stream. Outside, if a container takes >5s to cleanup due to Docker Desktop daemon load, the single `docker rm` attempt raises `RuntimeError('Scoring cleanup status is unknown')`, leaking the container.

Fix: Kill non-PID-1 descendants in `/proc` before printing the receipt (matching `DockerSandbox`). Add a retry loop to `docker rm --force`.

5. Documented Limitation Analysis: Shared-Interpreter Tampering

The packet acknowledges: *"Candidate Python and pytest currently share an interpreter... a sufficiently malicious Python program can tamper with the test runtime/receipt."* This is an inherent property of in-process Python unit testing.

Because tests must `import click`, the candidate module executes inside the same Python process evaluating the tests. The candidate can inspect interpreter memory (`gc.get_objects()`), locate the `Receipt` instance, or monkeypatch `_pytest.python.Function.runtest` to swallow all `AssertionError`s.

```
# UNEXECUTED — Demonstration of known limitation
import _pytest.python
_orig_runtest = _pytest.python.Function.runtest
def _always_pass(self):
    try: _orig_runtest(self)
    except AssertionError: return  # Swallow all test failures!
_pytest.python.Function.runtest = _always_pass
def add(a, b): return a - b
```

JSON schema validation or internal checksums cannot stop this, because the genuine `Receipt` plugin itself generates the valid receipt after the candidate swallows the assertion. This system provides strict filesystem/workspace isolation, but requires a post-hoc static AST scan on candidate diffs to catch deliberate evaluation evasion.

6. Architecture Decision: Improving Integrity

Dimension	Approach 1: Hardened In-Process (Recommended)	Approach 2: Privilege-Separated Container	Approach 3: Out-of-Process Oracle
Core Mechanism	Single container, single interpreter. Hardens OS FDs (dup2) and structures diagnostics so candidate errors map to `failed`.	Supervisor (UID 0) extracts files, locks hidden tests, spawns unprivileged worker (UID 65534) via pipe.	Test runner in trusted process. Candidate code runs in separate worker accessed via IPC/RPC.
Threats Prevented	Accidental FD 1 stdout corruption, child process leaks, false `RuntimeError` mapping, empty-file patch bugs.	Worker cannot modify files owned by supervisor or forge OS state.	Prevents monkeypatching of `pytest` and `AssertionError` swallowing.
Unresolved	In-memory monkeypatching of `pytest` / `Receipt`.	**Does NOT prevent interpreter tampering** because tests must import candidate code in the worker.	Cannot run tests that mock objects, yield generators, or test C internals.
Windows + Linux Docker	**100% Feasible immediately.** Uses existing pinned images with zero overhead.	Feasible, but requires relaxing strict `--cap-drop` / UID flags.	**Infeasible.** Requires rewriting all 63 frozen test suites.
Compatibility	**Exact match** on all 63 frozen tasks. 100% test compatibility.	Compatible, but adds IPC complexity.	Breaks benchmark validity and frozen receipts.

Honest Assessment of Python Test Authentication

Authenticating arbitrary test results against malicious Python code is impossible without rewriting the tests themselves. Dojo should not claim cryptographic tamper-resistance against Python runtime attacks. Instead, it offers a verifiable **Workspace Boundary Guarantee** (solvers never see hidden tests or contracts) and a **Stream Isolation Guarantee** (accidental candidate side-effects are cleanly classified). Deliberate evasion must be caught via post-hoc AST scanning of the candidate diffs.

7. Recommended Staged Roadmap

Stage A — Proven Defects (Pre-Registration Blocker)

- **OS-Level FD Isolation:** Duplicate FD 1 (`os.dup`), redirect FDs 1 and 2 to `/dev/null` via `os.dup2` , and write the receipt exclusively to the non-inheritable duplicated FD to block candidate output corruption.
- **Candidate Failure Classification:** Catch candidate-induced `pip install` failures and uncaught collection errors. Update the scoring logic to classify these as `failed` instead of `invalid_evidence` or `RuntimeError` , reserving crashes strictly for Docker/host infrastructure failures.
- **Empty-File Support:** Update `make_patch` to bypass `difflib` for empty file transitions (using standard `/dev/null` headers) and cleanly catch `UnicodeDecodeError` .
- **Descendant Cleanup:** Kill lingering non-PID-1 processes in `/proc` inside the container before emitting the receipt, and retry `docker rm --force` .

Stage B — Necessary Evaluation-Policy Decisions

- **Source Edit Policy Audit:** Run a one-time script across all 63 frozen tasks to verify that no non-test files modified by the reference commits are blocked by `editable_source` . For example, ensure `ChangedFile.is_test` does not false-positive on core library modules like `src/click/testing.py` .
- **Owner Implementation:** Replace `prepare_candidate_reference` with the owner's hand-written learning exercise code once completed.

Frozen File Policy for Implementation

The saved development smoke evidence (`runner-smoke-v1.json`) pins the SHA-256 hashes of the v1 evaluation files. To preserve this frozen evidence without invalidation, the implementation proposal provides a new module (`dojo/eval/docker_scoring_v2.py`) and a new test suite (`tests/test_contract_docker_v2.py`) that use the existing interfaces. If the primary engineer prefers an in-place update before committing, a minimal diff is also provided.

8. Implementation Proposal (v2 Module - Part 1)

File: `dojo/eval/docker_scoring_v2.py` (UNEXECUTED)

```
 import difflib, hashlib, inspect, json, math, uuid
from dataclasses import dataclass, field
from dojo.backends import local_docker as legacy
from dojo.backends.receipt_v2 import ReceiptV2
from dojo.eval.scoring_contract import PROFILES, ReceiptDecision, editable_source, judge_receipt
from dojo.mine.validate import snapshot_archive
from dojo.solve.docker_sandbox import DOCKER, PATCH_BYTES, PRELUDE, _bounded_run, _valid_name,
files_from_archive

FD_ISOLATION = r'''
_devnull = open(os.devnull, 'w+b')
_receipt_fd = os.dup(1)
os.set_inheritable(_receipt_fd, False)
os.dup2(_devnull.fileno(), 1)
os.dup2(_devnull.fileno(), 2)
sys.stdout = open(_devnull.fileno(), 'w', closefd=False)
sys.stderr = open(_devnull.fileno(), 'w', closefd=False)
'''

INSTALL_V2 = r'''
_install_failed = False
if control['install_project']:
    try:
        subprocess.run([sys.executable, '-I', '-m', 'pip', 'install', '--no-deps',
            '--no-build-isolation', '--no-index', '--no-cache-dir',
            '--disable-pip-version-check', '--target=/tmp/site', str(root)],
            check=True, stdin=subprocess.DEVNULL, stdout=_devnull, stderr=_devnull, timeout=8)
    except subprocess.CalledProcessError:
        _install_failed = True
    sys.path.insert(0, '/tmp/site')
    os.environ['PYTHONPATH'] = '/tmp/work/src:/tmp/work:/tmp/site'
    os.environ['PATH'] = '/tmp/site/bin:' + os.environ.get('PATH', '')
import pytest
if pytest.__version__ != '9.1.1': raise RuntimeError('Unexpected pytest version')
sys.path.insert(0, str(root))
if (root / 'src').is_dir(): sys.path.insert(0, str(root / 'src'))
'''

RUN_V2 = r'''
if _install_failed: code = 2
else:
    try:
        code = int(pytest.main(['-qq', '-c', '/dev/null', '--rootdir=/tmp/work',
            '-p', 'no:cacheprovider', '--confcutdir=/tmp/work', '--override-ini=addopts=',
            *control['selectors']], plugins=[receipt]))
    except SystemExit as exc:
        code = exc.code if isinstance(exc.code, int) and exc.code != 0 else 2
for _entry in pathlib.Path('/proc').iterdir():
    if _entry.name.isdigit() and int(_entry.name) not in (1, os.getpid()):
        try: os.kill(int(_entry.name), signal.SIGKILL)
        except ProcessLookupError: pass
'''

@dataclass(frozen=True)
class DockerScoreV2:
    status: str
    reason: str
    image_id: str
```

9. Implementation Proposal (v2 Module - Part 2)

```
def make_patch_v2(base, current):
    result, size = [], 0
    for name in sorted(base.keys() | current.keys()):
        in_base, in_curr = name in base, name in current
        old, new = base.get(name, b""), current.get(name, b"")
        if old == new and in_base == in_curr: continue
        if any(char.isspace() or char == '"' for char in name):
            raise ValueError("Quoted patch paths need a later patch exporter")
        if b"\0" in old or b"\0" in new: raise ValueError("Binary patch export is unsupported")
        try:
            old_lines = old.decode("utf-8").splitlines(keepends=True)
            new_lines = new.decode("utf-8").splitlines(keepends=True)
        except UnicodeDecodeError as exc:
            raise ValueError("Non-UTF-8 text patch export is unsupported") from exc
        fromfile, tofile = "a/" + name if in_base else "/dev/null", "b/" + name if in_curr else
"/dev/null"
        if not old and not new: lines = [f"--- {fromfile}\n", f"+++ {tofile}\n"]
        else: lines = list(difflib.unified_diff(old_lines, new_lines, fromfile=fromfile,
tofile=tofile))
        for line in lines:
            if not line.endswith("\n"): line += "\n\\ No newline at end of file\n"
            size += len(line.encode())
            if size > PATCH_BYTES: raise ValueError("Patch export exceeds adapter limits")
            result.append(line)
    return "".join(result)

def prepare_candidate_v2(contract, candidate_archive, patch):
    original, candidate = files_from_archive(contract.original_archive),
files_from_archive(candidate_archive)
    if not isinstance(patch, str) or make_patch_v2(original, candidate) != patch:
        raise ValueError("Candidate snapshot and patch disagree")
    restored = files_from_archive(contract.restored_archive)
    for name in original.keys() | candidate.keys():
        if editable_source(contract, name):
            if name in candidate: restored[name] = candidate[name]
            else: restored.pop(name, None)
    archive = snapshot_archive(restored)
    files_from_archive(archive)
    return archive

def harness_v2(protocol):
    if protocol == 'legacy-v1':
        _, found, tail = legacy.RUNNER.partition('class Receipt:\n')
        collector, boundary, _ = tail.partition('receipt = Receipt()\n')
        if not found or not boundary: raise RuntimeError('Frozen legacy collector layout changed')
        return (PRELUDE + FD_ISOLATION + INSTALL_V2 + 'class Receipt:\n' + collector
                + 'receipt = Receipt()\n' + RUN_V2 + r'''
if _install_failed: receipt.errors.append('candidate_offline_install_failed')
report = dict(exit_code=code, collected=receipt.collected, passed=sorted(receipt.passed),
    failed=sorted(receipt.failed), skipped=sorted(receipt.skipped), errors=sorted(receipt.errors))
os.write(_receipt_fd, ('DOJO_CONTRACT_RECEIPT=' + json.dumps(report) + '\n').encode('utf-8'))
''')
    if protocol == 'method-subtests-v2':
        return (PRELUDE + FD_ISOLATION + INSTALL_V2 + inspect.getsource(ReceiptV2)
                + '\nreceipt = ReceiptV2()\n' + RUN_V2 + r'''
if _install_failed: receipt.errors.add('candidate_offline_install_failed')
os.write(_receipt_fd, ('DOJO_CONTRACT_RECEIPT=' + json.dumps(receipt.report(code)) +
'\n').encode('utf-8'))
''')
    raise ValueError('Unreviewed scoring collector')
```

10. Implementation Proposal (v2 Module - Part 3)

```
def judge_candidate_receipt(contract, report, *, image_id):
    decision = judge_receipt(contract, report, image_id=image_id)
    if decision.status != "invalid_evidence": return decision
    if decision.reason not in {"runner_exit_or_collection", "collection_identity",
"exit_outcome_mismatch"}:
        return decision
    keys = {"exit_code", "collected", "passed", "failed", "skipped", "errors"}
    if contract.collector_protocol == "method-subtests-v2": keys |= {"collector_protocol",
"subtests"}
    if not isinstance(report, dict) or set(report) != keys: return decision
    if any(not isinstance(report[k], list) or any(not isinstance(n, str) for n in report[k])
           for k in ("passed", "failed", "skipped", "errors")): return decision
    if type(report["exit_code"]) is not int or type(report["collected"]) is not int or
report["collected"] < 0:
        return decision
    if "candidate_offline_install_failed" in report["errors"]:
        return ReceiptDecision("failed", "candidate_install_failed")
    if report["errors"] or report["exit_code"] == 2:
        return ReceiptDecision("failed", "collection_or_setup_error")
    return decision

def run_development_score_v2(contract, candidate_archive, patch, *, timeout=60):
    archive = prepare_candidate_v2(contract, candidate_archive, patch)
    name = 'dojo-contract-score-' + uuid.uuid4().hex
    command = [*DOCKER, 'run', '--name', name, '--pull=never', '--network=none', '--read-only',
               '--cap-drop=ALL', '--security-opt=no-new-privileges', '--user=65534:65534',
               f'--pids-limit={64 if contract.collector_protocol == "legacy-v1" else 128}',
               '--memory=256m', '--cpus=1', '--tmpfs=/tmp:rw,noexec,nosuid,nodev,size=64m',
               '--workdir=/tmp', '--no-healthcheck', '--entrypoint=/usr/local/bin/python',
               '-i', contract.image_id, '-I', '-c', harness_v2(contract.collector_protocol)]
    control = {'selectors': contract.selectors, 'install_project': contract.collector_protocol ==
'legacy-v1'}
    try:
        result = _bounded_run(command, timeout=timeout, output_limit=1024*1024,
                              input_data=json.dumps(control).encode() + b'\n' + archive)
        if result.returncode: raise RuntimeError('Scoring container or offline installation
failed')
        prefix = b'DOJO_CONTRACT_RECEIPT='
        lines = result.stdout.splitlines()
        if len(lines) != 1 or not lines[0].startswith(prefix):
            raise RuntimeError('Missing, noisy or ambiguous scoring receipt')
        report = json.loads(lines[0][len(prefix):])
        if "candidate_offline_install_failed" in report.get("errors", ()) and archive ==
contract.restored_archive:
            raise RuntimeError('Scoring container or offline installation failed') # Trusted
baseline broken
        decision = judge_candidate_receipt(contract, report, image_id=contract.image_id)
        return DockerScoreV2(decision.status, decision.reason, contract.image_id,
                             hashlib.sha256(command[-1].encode()).hexdigest(),
                             hashlib.sha256(archive).hexdigest(), report)
    finally:
        cleanup_err = None
        for _ in range(2):
            try: cleanup = _bounded_run([*DOCKER, 'rm', '--force', name], timeout=5,
output_limit=4096)
            except Exception as exc: cleanup_err = exc; continue
            if not cleanup.returncode or b'No such container' in cleanup.stderr: cleanup_err =
None; break
            cleanup_err = RuntimeError('Scoring container cleanup failed; stop')
        if cleanup_err: raise RuntimeError('Scoring cleanup status is unknown; stop') from
```

11. Pytest Regression Suite (Part 1)

File: `tests/test_contract_docker_v2.py` (UNEXECUTED)

```
 import json, subprocess, sys
import pytest
from dojo.eval import docker_scoring_v2 as runner_v2
from dojo.eval.contract_development import FIXED, TEST_FILE, fixture
from dojo.mine.validate import snapshot_archive
from dojo.solve.docker_sandbox import ProcessResult

@pytest.fixture(params=['pallets/click', 'more-itertools/more-itertools'])
def task(request): return fixture(request.param)

def test_make_patch_v2_supports_empty_file_creation_and_deletion(task):
    contract, source, base = task
    empty_subpkg = contract.source_prefix + 'subpkg/__init__.py'
    added = {**base, source: FIXED, empty_subpkg: b''}
    patch_add = runner_v2.make_patch_v2(base, added)
    assert f'--- /dev/null\n+++ b/{empty_subpkg}\n' in patch_add
    prepared = runner_v2.prepare_candidate_v2(contract, snapshot_archive(added), patch_add)
    assert empty_subpkg in runner_v2.files_from_archive(prepared)
    assert f'--- a/{empty_subpkg}\n+++ /dev/null\n' in runner_v2.make_patch_v2(added, base)

def test_make_patch_v2_rejects_non_utf8_bytes_with_value_error(task):
    _, source, base = task
    with pytest.raises(ValueError, match='Non-UTF-8'):
        runner_v2.make_patch_v2(base, {**base, source: b'\xff\xfeinvalid\n'})

def test_candidate_collection_or_syntax_error_scores_as_failed(task):
    contract, source, _ = task
    report = {'exit_code': 2, 'collected': 0, 'passed': [], 'failed': [], 'skipped': [], 'errors':
[TEST_FILE]}
    if contract.collector_protocol == 'method-subtests-v2':
        report.update(collector_protocol='method-subtests-v2', subtests={})
    decision = runner_v2.judge_candidate_receipt(contract, report, image_id=contract.image_id)
    assert decision.status == 'failed'
    assert decision.reason == 'collection_or_setup_error'

def test_candidate_fixture_setup_error_scores_as_failed(task):
    contract, _, _ = task
    report = {'exit_code': 1, 'collected': 2, 'passed': [], 'failed': [], 'skipped': [], 'errors':
['test_node']}
    if contract.collector_protocol == 'method-subtests-v2':
        report.update(collector_protocol='method-subtests-v2', subtests={})
    decision = runner_v2.judge_candidate_receipt(contract, report, image_id=contract.image_id)
    assert decision.status == 'failed'
    assert decision.reason == 'collection_or_setup_error'
```

12. Pytest Regression Suite (Part 2)

```
 def test_candidate_offline_install_failure_scores_failed_while_trusted_failure_raises(task,
monkeypatch):
    contract, source, base = task
    report = {'exit_code': 2, 'collected': 0, 'passed': [], 'failed': [], 'skipped': [],
              'errors': ['candidate_offline_install_failed']}
    if contract.collector_protocol == 'method-subtests-v2':
report.update(collector_protocol='method-subtests-v2', subtests={})

    def fake_run(command, **kwargs):
        if 'inspect' in command: return ProcessResult(0, contract.image_id.encode(), b'')
        if 'rm' in command: return ProcessResult(0, b'', b'')
        return ProcessResult(0, b'DOJO_CONTRACT_RECEIPT=' + json.dumps(report).encode() + b'\n',
b'')
    monkeypatch.setattr(runner_v2, '_bounded_run', fake_run)

    deleted = {k: v for k, v in base.items() if k != source}
    score = runner_v2.run_development_score_v2(contract, snapshot_archive(deleted),
runner_v2.make_patch_v2(base, deleted))
    assert score.status == 'failed'
    assert score.reason == 'candidate_install_failed'

    with pytest.raises(RuntimeError, match='offline installation failed'):
        runner_v2.run_development_score_v2(contract, snapshot_archive(base), '')

def test_harness_v2_redirects_os_fd1_so_low_level_stdout_cannot_corrupt_receipt(task):
    contract, _, _ = task
    code = runner_v2.harness_v2(contract.collector_protocol)
    assert '_receipt_fd = os.dup(1)' in code
    probe = (
        "import os, subprocess, sys\n" + runner_v2.FD_ISOLATION +
        "os.write(1, b'OS_FD1_NOISE\\n')\nprint('PYTHON_STDOUT_NOISE')\n" +
        "subprocess.run([sys.executable, '-c', 'import os; os.write(1,
b\"CHILD_NOISE\\\\n\")'])\n" +
        "os.write(_receipt_fd, b'DOJO_CONTRACT_RECEIPT={\"ok\": 1}\\n')\n"
    )
    proc = subprocess.run([sys.executable, "-I", "-c", probe], capture_output=True, check=True,
timeout=5)
    assert json.loads(proc.stdout.splitlines()[0][len(b'DOJO_CONTRACT_RECEIPT='):]) == {"ok": 1}

def test_transient_cleanup_failure_retries_and_succeeds(task, monkeypatch):
    contract, source, base = task
    rm_attempts = 0
    report = {'exit_code': 0, 'collected': 1, 'passed': ['t'], 'failed': [], 'skipped': [],
'errors': []}
    if contract.collector_protocol == 'method-subtests-v2': report.update(collector_protocol='v2',
subtests={})

    def fake_run(command, **kwargs):
        nonlocal rm_attempts
        if 'inspect' in command: return ProcessResult(0, contract.image_id.encode(), b'')
        if 'rm' in command:
            rm_attempts += 1
            return ProcessResult(1, b'', b'busy') if rm_attempts == 1 else ProcessResult(0, b'',
b'')
        return ProcessResult(0, b'DOJO_CONTRACT_RECEIPT=' + json.dumps(report).encode() + b'\n',
b'')
    monkeypatch.setattr(runner_v2, '_bounded_run', fake_run)
    files = {**base, source: FIXED}
    runner_v2.run_development_score_v2(contract, snapshot_archive(files),
```

13. Acceptance and Integration Plan

Check Type	Action / Command	Expected Outcome	Blocking Condition
1. Frozen Integrity	Run `scoring_contract check` & `pytest tests/test_contract_docker.py`.	63 frozen contracts verify. Existing v1 smoke tests pass.	Hash mismatch in task records or manifests.
2. Edit-Policy Audit	Verify all non-test files changed in 63 tasks pass `editable_source`.	100% of reference edits pass policy without false blocks.	Legitimate non-test source edits blocked.
3. Hardened Unit Suite	Run `pytest tests/test_contract_docker_v2.py`.	Empty-file patches succeed, FD isolation works, collection errors return `failed`.	Any failure in `test_contract_docker_v2.py`.
4. Synthetic Containers	Run matrix across 13 cases (fixed, broken, noisy, timeout).	Cases pass/fail accurately. No `RuntimeError` on candidate bugs. Zero leaked containers.	Leaked containers, false `invalid_evidence`, or false crashes.
5. Evidence Checkpoint	Write `runner-smoke-v2.json` with new SHA-256 hashes.	JSON file created with 0 provider calls.	Non-zero provider calls.

Residual Risks

Shared-Interpreter Tampering (Known Limitation): A candidate that monkeypatches `_pytest` can forge passing outcomes. Mitigate before benchmark publication via a post-hoc static AST check on candidate diffs. **legacy-v1 Teardown Skip:** Frozen `legacy-v1` code double-counts tests that pass then skip in teardown, yielding `invalid_evidence`. (Use `method-subtests-v2` for new tasks).

Three Most Important Decisions

- **Candidate Error Classification (Finding 1):** Confirm that candidate-induced `pip install` and collection errors should be scored as `failed` (not `RuntimeError` or `invalid_evidence`).
- **In-Place vs. V2 Module:** Decide whether to update uncommitted runner files in place or keep v1 files untouched and use `docker_scoring_v2.py` to preserve frozen hash compatibility.
- **Pre-Registration Audit:** Before Stage B, confirm that `editable_source` does not block legitimate library modules (e.g. `src/click/testing.py`).

Specific Requests for Missing Source Excerpts

To verify the omitted stated assumptions, please provide:

1. **`dojo/mine/git_mine.py`:** `ChangedFile` and `is_test` logic.
2. **`dojo/mine/real_task.py`:** `safe_path`, `MAX_BYTES`, and `MAX_FILES`.
3. **`dojo/mine/validate.py`:** `snapshot_archive` tar normalization logic.
4. **`dojo/solve/input_guard_v2.py`:** `build_solver_input` and `_reject_substitution`.